

invented independently of the discovery of dynamic programming and only later came to be viewed as examples of this technique's applications.) Numerous other applications range from the optimal way of breaking text into lines (e.g., [Baa00]) to image resizing [Avi07] to a variety of applications to sophisticated engineering problems (e.g., [Ber01]).

8.1 Three Basic Examples

The goal of this section is to introduce dynamic programming via three typical examples.

EXAMPLE 1 *Coin-row problem* There is a row of n coins whose values are some positive integers $c_1, c_2, \dots, c_n$, not necessarily distinct. The goal is to pick up the maximum amount of money subject to the constraint that no two coins adjacent in the initial row can be picked up.

Let $F(n)$ be the maximum amount that can be picked up from the row of n coins. To derive a recurrence for $F(n)$, we partition all the allowed coin selections into two groups: those that include the last coin and those without it. The largest amount we can get from the first group is equal to $c_n + F(n - 2)$ —the value of the n th coin plus the maximum amount we can pick up from the first $n - 2$ coins. The maximum amount we can get from the second group is equal to $F(n - 1)$ by the definition of $F(n)$. Thus, we have the following recurrence subject to the obvious initial conditions:

$$\begin{aligned} F(n) &= \max\{c_n + F(n - 2), F(n - 1)\} \quad \text{for } n > 1, \\ F(0) &= 0, \quad F(1) = c_1. \end{aligned} \tag{8.3}$$

We can compute $F(n)$ by filling the one-row table left to right in the manner similar to the way it was done for the n th Fibonacci number by Algorithm *Fib*(n) in Section 2.5.

ALGORITHM *CoinRow*($C[1..n]$)

```
//Applies formula (8.3) bottom up to find the maximum amount of money
//that can be picked up from a coin row without picking two adjacent coins
//Input: Array  $C[1..n]$  of positive integers indicating the coin values
//Output: The maximum amount of money that can be picked up
 $F[0] \leftarrow 0; \quad F[1] \leftarrow C[1]$ 
for  $i \leftarrow 2$  to  $n$  do
     $F[i] \leftarrow \max(C[i] + F[i - 2], F[i - 1])$ 
return  $F[n]$ 
```

The application of the algorithm to the coin row of denominations 5, 1, 2, 10, 6, 2 is shown in Figure 8.1. It yields the maximum amount of 17. It is worth pointing

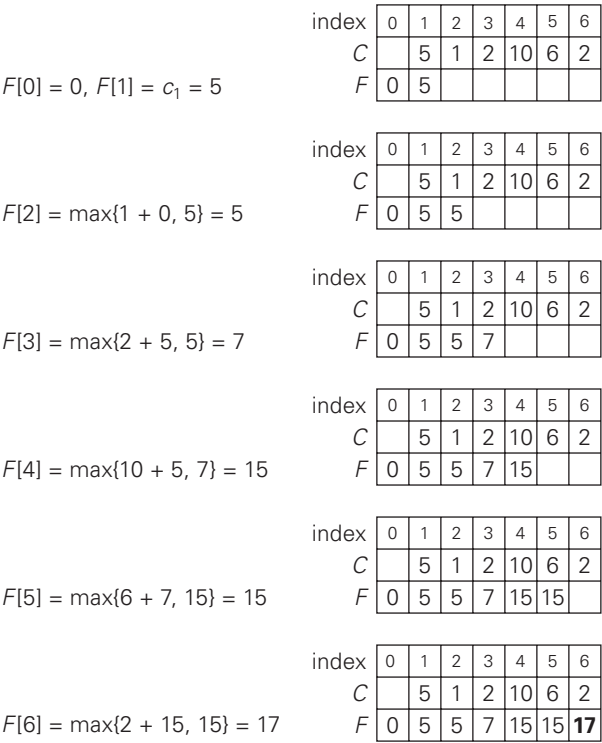


FIGURE 8.1 Solving the coin-row problem by dynamic programming for the coin row 5, 1, 2, 10, 6, 2.

out that, in fact, we also solved the problem for the first i coins in the row given for every $1 \leq i \leq 6$. For example, for $i = 3$, the maximum amount is $F(3) = 7$.

To find the coins with the maximum total value found, we need to backtrace the computations to see which of the two possibilities— $c_n + F(n - 2)$ or $F(n - 1)$ —produced the maxima in formula (8.3). In the last application of the formula, it was the sum $c_6 + F(4)$, which means that the coin $c_6 = 2$ is a part of an optimal solution. Moving to computing $F(4)$, the maximum was produced by the sum $c_4 + F(2)$, which means that the coin $c_4 = 10$ is a part of an optimal solution as well. Finally, the maximum in computing $F(2)$ was produced by $F(1)$, implying that the coin c_2 is not the part of an optimal solution and the coin $c_1 = 5$ is. Thus, the optimal solution is $\{c_1, c_4, c_6\}$. To avoid repeating the same computations during the backtracing, the information about which of the two terms in (8.3) was larger can be recorded in an extra array when the values of F are computed.

Using the *CoinRow* to find $F(n)$, the largest amount of money that can be picked up, as well as the coins composing an optimal set, clearly takes $\Theta(n)$ time and $\Theta(n)$ space. This is by far superior to the alternatives: the straightforward top-

down application of recurrence (8.3) and solving the problem by exhaustive search (Problem 3 in this section's exercises). ■

EXAMPLE 2 *Change-making problem* Consider the general instance of the following well-known problem. Give change for amount n using the minimum number of coins of denominations $d_1 < d_2 < \dots < d_m$. For the coin denominations used in the United States, as for those used in most if not all other countries, there is a very simple and efficient algorithm discussed in the next chapter. Here, we consider a dynamic programming algorithm for the general case, assuming availability of unlimited quantities of coins for each of the m denominations $d_1 < d_2 < \dots < d_m$ where $d_1 = 1$.

Let $F(n)$ be the minimum number of coins whose values add up to n ; it is convenient to define $F(0) = 0$. The amount n can only be obtained by adding one coin of denomination d_j to the amount $n - d_j$ for $j = 1, 2, \dots, m$ such that $n \geq d_j$. Therefore, we can consider all such denominations and select the one minimizing $F(n - d_j) + 1$. Since 1 is a constant, we can, of course, find the smallest $F(n - d_j)$ first and then add 1 to it. Hence, we have the following recurrence for $F(n)$:

$$\begin{aligned} F(n) &= \min_{j: n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0, \\ F(0) &= 0. \end{aligned} \tag{8.4}$$

We can compute $F(n)$ by filling a one-row table left to right in the manner similar to the way it was done above for the coin-row problem, but computing a table entry here requires finding the minimum of up to m numbers.

ALGORITHM *ChangeMaking*($D[1..m], n$)

```
//Applies dynamic programming to find the minimum number of coins
//of denominations  $d_1 < d_2 < \dots < d_m$  where  $d_1 = 1$  that add up to a
//given amount  $n$ 
//Input: Positive integer  $n$  and array  $D[1..m]$  of increasing positive
//      integers indicating the coin denominations where  $D[1] = 1$ 
//Output: The minimum number of coins that add up to  $n$ 
 $F[0] \leftarrow 0$ 
for  $i \leftarrow 1$  to  $n$  do
     $temp \leftarrow \infty$ ;  $j \leftarrow 1$ 
    while  $j \leq m$  and  $i \geq D[j]$  do
         $temp \leftarrow \min(F[i - D[j]], temp)$ 
         $j \leftarrow j + 1$ 
     $F[i] \leftarrow temp + 1$ 
return  $F[n]$ 
```

The application of the algorithm to amount $n = 6$ and denominations 1, 3, 4 is shown in Figure 8.2. The answer it yields is two coins. The time and space efficiencies of the algorithm are obviously $O(nm)$ and $\Theta(n)$, respectively.

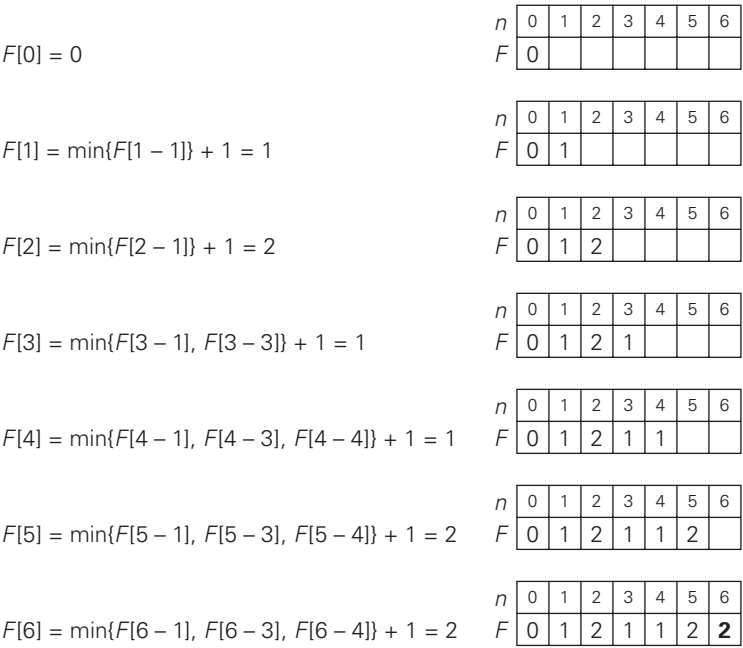


FIGURE 8.2 Application of Algorithm *MinCoinChange* to amount $n = 6$ and coin denominations 1, 3, and 4.

To find the coins of an optimal solution, we need to backtrack the computations to see which of the denominations produced the minima in formula (8.4). For the instance considered, the last application of the formula (for $n = 6$), the minimum was produced by $d_2 = 3$. The second minimum (for $n = 6 - 3$) was also produced for a coin of that denomination. Thus, the minimum-coin set for $n = 6$ is two 3's. ■

EXAMPLE 3 *Coin-collecting problem* Several coins are placed in cells of an $n \times m$ board, no more than one coin per cell. A robot, located in the upper left cell of the board, needs to collect as many of the coins as possible and bring them to the bottom right cell. On each step, the robot can move either one cell to the right or one cell down from its current location. When the robot visits a cell with a coin, it always picks up that coin. Design an algorithm to find the maximum number of coins the robot can collect and a path it needs to follow to do this.

Let $F(i, j)$ be the largest number of coins the robot can collect and bring to the cell (i, j) in the i th row and j th column of the board. It can reach this cell either from the adjacent cell $(i - 1, j)$ above it or from the adjacent cell $(i, j - 1)$ to the left of it. The largest numbers of coins that can be brought to these cells are $F(i - 1, j)$ and $F(i, j - 1)$, respectively. Of course, there are no adjacent cells

above the cells in the first row, and there are no adjacent cells to the left of the cells in the first column. For those cells, we assume that $F(i - 1, j)$ and $F(i, j - 1)$ are equal to 0 for their nonexistent neighbors. Therefore, the largest number of coins the robot can bring to cell (i, j) is the maximum of these two numbers plus one possible coin at cell (i, j) itself. In other words, we have the following formula for $F(i, j)$:

$$\begin{aligned} F(i, j) &= \max\{F(i - 1, j), F(i, j - 1)\} + c_{ij} \quad \text{for } 1 \leq i \leq n, \quad 1 \leq j \leq m \\ F(0, j) &= 0 \quad \text{for } 1 \leq j \leq m \quad \text{and} \quad F(i, 0) = 0 \quad \text{for } 1 \leq i \leq n, \end{aligned} \quad (8.5)$$

where $c_{ij} = 1$ if there is a coin in cell (i, j) , and $c_{ij} = 0$ otherwise.

Using these formulas, we can fill in the $n \times m$ table of $F(i, j)$ values either row by row or column by column, as is typical for dynamic programming algorithms involving two-dimensional tables.

ALGORITHM *RobotCoinCollection*($C[1..n, 1..m]$)

```
//Applies dynamic programming to compute the largest number of
//coins a robot can collect on an  $n \times m$  board by starting at (1, 1)
//and moving right and down from upper left to down right corner
//Input: Matrix  $C[1..n, 1..m]$  whose elements are equal to 1 and 0
//for cells with and without a coin, respectively
//Output: Largest number of coins the robot can bring to cell  $(n, m)$ 
 $F[1, 1] \leftarrow C[1, 1]$ ; for  $j \leftarrow 2$  to  $m$  do  $F[1, j] \leftarrow F[1, j - 1] + C[1, j]$ 
for  $i \leftarrow 2$  to  $n$  do
     $F[i, 1] \leftarrow F[i - 1, 1] + C[i, 1]$ 
    for  $j \leftarrow 2$  to  $m$  do
         $F[i, j] \leftarrow \max(F[i - 1, j], F[i, j - 1]) + C[i, j]$ 
return  $F[n, m]$ 
```

The algorithm is illustrated in Figure 8.3b for the coin setup in Figure 8.3a. Since computing the value of $F(i, j)$ by formula (8.5) for each cell of the table takes constant time, the time efficiency of the algorithm is $\Theta(nm)$. Its space efficiency is, obviously, also $\Theta(nm)$.

Tracing the computations backward makes it possible to get an optimal path: if $F(i - 1, j) > F(i, j - 1)$, an optimal path to cell (i, j) must come down from the adjacent cell above it; if $F(i - 1, j) < F(i, j - 1)$, an optimal path to cell (i, j) must come from the adjacent cell on the left; and if $F(i - 1, j) = F(i, j - 1)$, it can reach cell (i, j) from either direction. This yields two optimal paths for the instance in Figure 8.3a, which are shown in Figure 8.3c. If ties are ignored, one optimal path can be obtained in $\Theta(n + m)$ time.

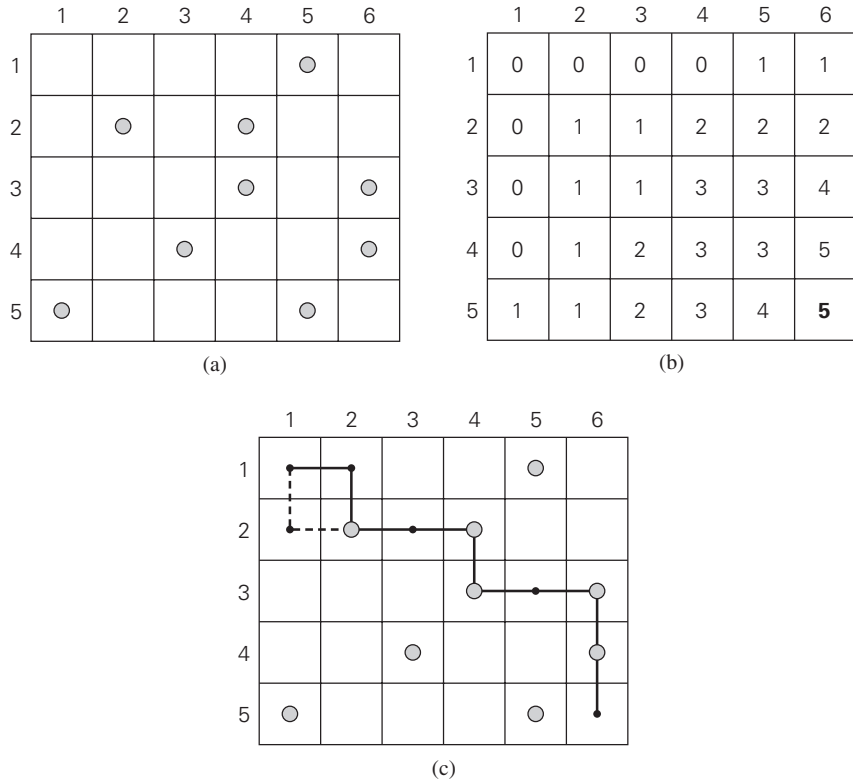


FIGURE 8.3 (a) Coins to collect. (b) Dynamic programming algorithm results. (c) Two paths to collect 5 coins, the maximum number of coins possible.

Exercises 8.1

1. What does dynamic programming have in common with divide-and-conquer? What is a principal difference between them?
2. Solve the instance 5, 1, 2, 10, 6 of the coin-row problem.
3. **a.** Show that the time efficiency of solving the coin-row problem by straight-forward application of recurrence (8.3) is exponential.
b. Show that the time efficiency of solving the coin-row problem by exhaustive search is at least exponential.
4. Apply the dynamic programming algorithm to find all the solutions to the change-making problem for the denominations 1, 3, 5 and the amount $n = 9$.